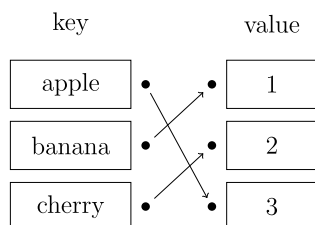


Hash

哈希表

引入



哈希表又称散列表，一种以「key-value」形式存储数据的数据结构。所谓以「key-value」形式存储数据，是指任意的键值 key 都唯一对应到内存中的某个位置。只需要输入查找的键值，就可以快速地找到其对应的 value。可以把哈希表理解为一种高级的数组，这种数组的下标可以是很大的整数，浮点数，字符串甚至结构体。

哈希函数

要让键值对应到内存中的位置，就要为键值计算索引，也就是计算这个数据应该放到哪里。这个根据键值计算索引的函数就叫做哈希函数，也称散列函数。举个例子，如果键值是一个人的身份证号码，哈希函数就可以是号码的后四位，当然也可以是号码的前四位。生活中常用的「手机尾号」也是一种哈希函数。在实际的应用中，键值可能是更复杂的东西，比如浮点数、字符串、结构体等，这时候就要根据具体情况设计合适的哈希函数。哈希函数应当易于计算，并且尽量使计算出来的索引均匀分布。

能为 key 计算索引之后，我们就可以知道每个键值对应的值 value 应该放在哪里了。假设我们用数组 a 存放数据，哈希函数是 f，那键值对 (key, value) 就应该放在 $a[f(key)]$ 上。不论键值是什么类型，范围有多大， $f(key)$ 都是在可接受范围内的整数，可以作为数组的下标。

在 OI 中，最常见的情况应该是键值为整数的情况。当键值的范围比较小的时候，可以直接把键值作为数组的下标，但当键值的范围比较大，比如以 范围内的整数作为键值的时候，就需要用到哈希表。一般把键值模一个较大的质数作为索引，也就是取 作为哈希函数。

另一种比较常见的情况是 key 为字符串的情况，由于不支持以字符串作为数组下标，并且将字符串转化成数字存储也可以避免多次进行字符串比较。所以在 OI 中，一般不直接把字符串作为键值，而是先算出字符串的哈希值，再把其哈希值作为键值插入到哈希表里。关于字符串的哈希值，我们一般采用进制的思想，将字符串想象成一个 进制的数。那么，对于每一个长度为 的字符串，就有：

我们可以将得到的 对 （即 unsigned long long 的最大值）取模。这样 unsigned long long 的自然溢出就等价于取模操作了。可以使操作更加方便。

这种方法虽然简单，但并不是完美的。可以构造数据使这种方法发生冲突（即两个字符串的 对 取模后的结果相同）。

我们可以使用双哈希的方法：选取两个大质数 。当且仅当两个字符串的哈希值对 和对 取模都相等时，我们才认为这两个字符串相等。这样可以大大降低哈希冲突的概率。

冲突

如果对于任意的键值，哈希函数计算出来的索引都不相同，那只用根据索引把 (key, value) 放到对应的位置就行了。但实际上，常常会出现两个不同的键值，他们用哈希函数计算出来的索引是相同的。这时候就需要一些方法来处理冲突。在 OI 中，最常用的方法是拉链法。

拉链法

拉链法也称开散列法 (open hashing)。

拉链法是在每个存放数据的地方开一个链表，如果有多个键值索引到同一个地方，只用把他们都放到那个位置的链表里就行了。查询的时候需要把对应位置的链表整个扫一遍，对其中的每个数据比较其键值与查询的键值是否一致。如果索引的范围是 $[0, M)$ ，哈希表的大小为 N ，那么一次插入/查询需要进行期望 $\frac{N}{M}$ 次比较。

实现



C++Python

```
1 constexpr int SIZE = 1000000;
2 constexpr int M = 999997;
3
4 struct HashTable {
5     struct Node {
6         int next, value, key;
7     } data[SIZE];
8
9     int head[M], size;
10
11     int f(int key) { return (key % M + M) % M; }
12
13     int get(int key) {
14         for (int p = head[f(key)]; p; p = data[p].next)
15             if (data[p].key == key) return data[p].value;
16         return -1;
17     }
18
19     int modify(int key, int value) {
20         for (int p = head[f(key)]; p; p = data[p].next)
21             if (data[p].key == key) return data[p].value = value;
22     }
23
24     int add(int key, int value) {
25         if (get(key) != -1) return -1;
26         data[++size] = Node{head[f(key)], value, key};
27         head[f(key)] = size;
28         return value;
29     }
30 };
```

```

1 M = 999997
2 SIZE = 1000000
3
4
5 class Node:
6     def __init__(self, next=None, value=None, key=None):
7         self.next = next
8         self.value = value
9         self.key = key
10
11
12 data = [Node() for _ in range(SIZE)]
13 head = [0] * M
14 size = 0
15
16
17 def f(key):
18     return key % M
19
20
21 def get(key):
22     p = head[f(key)]
23     while p:
24         if data[p].key == key:
25             return data[p].value
26         p = data[p].next
27     return -1
28
29
30 def modify(key, value):
31     p = head[f(key)]
32     while p:
33         if data[p].key == key:
34             data[p].value = value
35             return data[p].value
36         p = data[p].next
37
38
39 def add(key, value):
40     if get(key) != -1:
41         return -1
42     size = size + 1
43     data[size] = Node(head[f(key)], value, key)
44     head[f(key)] = size
45     return value

```

这里再提供一个封装过的模板，可以像 map 一样用，并且较短

```

1 struct hash_map { // 哈希表模板
2
3     struct data {
4         long long u;
5         int v, nex;
6     }; // 前向星结构
7
8     data e[SZ << 1]; // SZ 是 const int 表示大小
9     int h[SZ], cnt;
10
11     int hash(long long u) { return (u % SZ + SZ) % SZ; }
12
13     // 这里使用 (u % SZ + SZ) % SZ 而非 u % SZ 的原因是
14     // C++ 中的 % 运算无法将负数转为正数
15
16     int& operator[] (long long u) {
17         int hu = hash(u); // 获取头指针
18         for (int i = h[hu]; i; i = e[i].nex)
19             if (e[i].u == u) return e[i].v;
20         return e[++cnt] = data{u, -1, h[hu]}, h[hu] = cnt, e[cnt].v;
21     }
22
23     hash_map() {
24         cnt = 0;
25         memset(h, 0, sizeof(h));
26     }
27 };

```

在这里，hash 函数是针对键值的类型设计的，并且返回一个链表头指针用于查询。在这个模板中我们写了一个键值对类型为 (long long, int) 的 hash 表，并且在查询不存在的键值时返回 -1。函数 hash_map() 用于在定义时初始化。

闭散列法

闭散列方法把所有记录直接存储在散列表中，如果发生冲突则根据某种方式继续进行探查。

比如线性探查法：如果在 d 处发生冲突，就依次检查 $d + 1$, $d + 2$

实现

```

1 constexpr int N = 360007; // N 是最大可以存储的元素数量
2
3 class Hash {
4 private:
5     int keys[N];
6     int values[N];
7
8 public:
9     Hash() { memset(values, 0, sizeof(values)); }
10
11     int& operator[] (int n) {
12         // 返回一个指向对应 Hash[Key] 的引用
13         // 修改成不为 0 的值 0 时候视为空
14         int idx = (n % N + N) % N, cnt = 1;
15         while (keys[idx] != n && values[idx] != 0) {
16             idx = (idx + cnt * cnt) % N;
17             cnt += 1;
18         }
19         keys[idx] = n;
20         return values[idx];
21     }
22 };

```

例题

[「JLOI2011」不重复数字](#)

本页面最近更新：2024/10/9 22:38:42, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [iamtwz](#), [CCXXI](#), [ChungZH](#), [Early0v0](#), [Enter-tainer](#), [HarumiKiyama](#), [Henry-ZHR](#), [HXLLL](#), [ImpleLee](#), [Ir1d](#), [ksyx](#), [lhxhxxxx](#), [littlefrogfromthenorth](#), [LTHAndy](#), [lyccrius](#), [mcendu](#), [memset0](#), [Menci](#), [opsiff](#), [ouuan](#), [shawlleyw](#), [sshwy](#), [StudyingFather](#), [Tiphereth-A](#), [WASSER2545](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用